

System & Performance Engineering for Industrial MCP Agents

Yassine Jebbouri Darief Maes Aishani Rachakonda Vivek Iyer Dhaval C Patel

Abstract—Industrial asset-operations agents must diagnose faults by invoking multiple domain-specific tool servers, each of which can itself call LLM APIs. We present a performance engineering study of a plan-execute agent built with MCP and benchmarked on the IBM Research AssetOpsBench dataset. A key bottleneck is the Failure Mode-Sensor Relevancy (FMSR) tool server, which executes up to 63 sequential LLM calls per scenario, producing wall times of 559-894s under default configurations. We investigate three orthogonal optimization tracks: (1) speculative hedged parallelization of the $N \times M$ relevancy matrix, (2) database context prefetching to lower memory movement latency, and (3) quantized model substitution for the binary classification sub-task. Hedged speculative execution reduces worst-case wall time by up to $36\times$ (559s to 15.5s) and aggregate mean wall time by $11.2\times$ (195.1s to 17.4s) across 15 heavy-mapping scenarios. Across 300 quantization runs spanning five open-source model families and three precision levels, 4-bit (INT4) quantized models are most optimal: highest tool-call accuracy (0.675), lowest memory (2.0GB), and latency within 0.30s of FP16. A separate direct-vs-MCP system experiment shows the FastMCP path gains 24.8 percentage points of LLM-judge accuracy over the in-process baseline (46.2% to 71.0%), a gain that traces to rescuing trajectories the in-process path fails on due to improper id-association. A 3B-parameter INT4 local model on CPU exceeds the 70B-class WatsonX cloud baseline by 1.5pp on tool-call accuracy while reducing memory by over 95%.

Index Terms—LLM agents, plan-execute, Model Context Protocol, quantization, speculative execution, industrial AI, AssetOpsBench

I. INTRODUCTION

Autonomous agents that diagnose industrial equipment faults must coordinate information retrieval, time-series forecasting, and domain-specific rule matching across heterogeneous systems [?]. MCP [?] standardizes tool invocation, letting agents compose pipeline steps across independently deployed servers. Each inter-server call can itself trigger further LLM inference, creating additional latency potential particularly when factoring in rate limits.

This paper studies one such pipeline: a plan-execute agent [4] orchestrating four MCP servers (IoT sensor retrieval, FMSR fault mapping, TSFM time-series forecasting, and Utilities) over the IBM Research AssetOpsBench [1] benchmark. Profiling reveals that the FMSR server’s `get_failure_mode_sensor_mapping` procedure dominates wall time by executing one LLM call per (sensor, failure mode) pair sequentially, causing up to 63 calls per scenario.

The contributions are as follows:

- A profiling methodology for MCP agent pipelines with per-phase hardware instrumentation (wall, CPU%, RSS, threads via psutil), monkey-patched LLM-call traces, and JSONL output.
- Four parallelization strategies for the $N \times M$ bottleneck: sequential, parallel, inverted AIMD (ceiling-start), and hedged speculative execution. Hedging dominates 14 of 15 scenarios in the strategy sweep with an aggregate $11.2\times$ mean speedup.
- A 300-run quantization study across five model families at three precision levels. INT4 is optimal in every family: highest accuracy, lowest memory, latency within 0.30s of FP16.
- An empirical demonstration that a 3B-parameter INT4 model running locally on CPU exceeds the 70B-class WatsonX cloud baseline on tool-call accuracy (0.675 vs 0.660), a counter-intuitive result whose mechanism we explain (though it is important to note this result still falls within error bounds for statistical significance).
- A direct-vs-MCP system experiment showing FastMCP gains 24.8pp of LLM-judge accuracy over the in-process baseline, traced to MCP’s error-as-data envelope and not to reasoning quality.

II. BACKGROUND AND RELATED WORK

A. Plan-Execute Agents

Plan-execute agents [4] separate task decomposition from execution: an LLM generates a structured multi-step plan, and a separate executor dispatches each step to the appropriate tool or sub-agent. This contrasts with ReAct [3], which interleaves reasoning and action in a single loop. Our system adopts plan-execute because it exposes the full tool dependency graph upfront, enabling step-level parallelism not possible in ReAct’s sequential think-act cycle.

B. LLM Quantization

Post-training quantization compresses model weights to lower bit-widths. LLM.int8() [5] demonstrated that 8-bit inference preserves perplexity for large models via mixed-precision decomposition. GPTQ [6] extended this to 4-bit via second-order weight updates. The Q4_K_M scheme used in `llama.cpp` [7] applies mixed-group 4-bit quantization, retaining higher precision for attention-critical weights.

C. Speculative and Hedged Execution

Dean and Barroso [8] introduced hedged requests (firing duplicate requests and using the first response) to cap tail latency in distributed systems. We apply this principle to LLM API calls, where latency is bimodal: calls typically complete in 1–2 s but stall for 30–90 s under rate limiting.

D. Adaptive Concurrency

AIMD [?] (additive-increase, multiplicative-decrease) was developed for TCP congestion control. The standard form sometimes pays a significant wall-time tax on smaller, all-or-nothing workloads: the additive-up phase often does not reach a local ceiling before the workload finishes. We invert it (start at the ceiling, halve on the first error).

III. SYSTEM ARCHITECTURE

A. Pipeline Overview

The agent pipeline follows four phases: DISCOVER, PLAN, EXECUTE, and SUMMARISE. During DISCOVER, the executor queries each registered MCP server for its tool manifest. During PLAN, an LLM (meta-llama/llama-3-3-70b-instruct via WatsonX) decomposes the user query into an ordered list of tool-call steps with explicit agent assignments. EXECUTE dispatches each step to its assigned MCP server over JSON-RPC, collects results, and injects them into subsequent steps. SUMMARISE condenses the step-by-step execution history into a final answer.

B. FMSR Server and the $N \times M$ Bottleneck

The FMSR server implements fault diagnosis via a relevancy matrix: for each (asset, failure mode) query, it determines which of the asset’s installed sensors can detect each failure mode by invoking an LLM classifier. For Chiller 6 at site MAIN — the primary asset in AssetOpsBench — this produces $9 \times 5 - 7 = 45 - 63$ LLM calls per scenario. In the sequential baseline, these calls execute one at a time, bounded by WatsonX’s 2 req/s free-tier limit.

The default `_call_relevancy` backend is `watsonx/meta-llama/llama-3-3-70b-instruct`, configurable via the `FMSR_MODEL_ID` environment variable routed through LiteLLM. All three optimization tracks leverage this.

C. Benchmark Harness

We implemented a benchmark harness (BenchmarkRunner) that sweeps (orchestrator, scenario, strategy) cells, logging wall time, per-phase duration, token usage, tool-call sequences, and hardware metrics (CPU%, RAM, disk I/O) to append-only JSONL files with filelock for crash-safe resumability. Tool-call accuracy is computed against the expected tool sequence from the HuggingFace `ibm-research/AssetOpsBench` dataset. Run information is logged to Weights & Biases for tracking.

IV. OPTIMIZATION METHODS

A. Parallelization with Speculative Hedging

1) *Baseline*: The sequential baseline issues relevancy calls one at a time via Python’s `asyncio` event loop, awaiting each before dispatching the next. Under WatsonX free-tier limits, a scenario with 63 calls takes $63 \times \bar{t}_{\text{call}} \approx 559\text{--}894$ s end-to-end.

2) *Parallel (Fix 3)*: We replace sequential dispatch with `asyncio.gather` over a `Semaphore(8)`, limiting concurrency to eight in-flight requests. Transient 429 errors are retried with exponential backoff managed by the LiteLLM Router. This reduces wall time approximately proportionally to the concurrency factor, subject to the 10-concurrent-request WatsonX cap.

3) *Adaptive Ceiling-Start*: Classical AIMD (Additive Increase, Multiplicative Decrease) ramps concurrency up from 1, incurring unnecessary ramp-up latency. We invert this: start at the concurrency ceiling (8), halve on 429 errors, and increment by 1 on success. This eliminates the cold-start penalty and is effective on scenarios with fewer than 20 calls.

4) *Hedged Speculative Requests*: WatsonX free-tier latency is bimodal: most calls return in 1–2 s, but a fraction stall for 30–90 s awaiting rate-limit release or backend scheduling. Sequential and parallel strategies wait out the full stall. We instead treat 8 s of silence as a stall signal and fire a duplicate request; whichever response arrives first is used and the other is cancelled (`asyncio.wait` with `FIRST_COMPLETED`). This caps p_{95} latency at approximately $2 \times t_{\text{hedge}} = 16$ s at the cost of $\sim 5\text{--}15\%$ additional token usage on stalled calls.

Algorithm 1 Hedged Relevancy Call

Require: asset a , failure mode f , sensor s , hedge timeout $\tau = 8$ s

- 1: $t_1 \leftarrow \text{ASYNCLLMCALL}(a, f, s)$
- 2: **await** `SLEEP( $\tau$ )`
- 3: $t_2 \leftarrow \text{ASYNCLLMCALL}(a, f, s)$ $\triangleright$ duplicate
- 4: $r, _ \leftarrow \text{WAIT}(\{t_1, t_2\}, \text{return_when=FIRST_COMPLETED})$
- 5: **cancel** remaining task
- 6: **return** $r.\text{result}()$

B. Database Context Prefetching

The planner issues IoT tool calls (sites, assets, sensors) at the start of each scenario to discover the asset configuration before decomposing the task. We hypothesized that prefetching this context once and injecting it into the planner prompt would eliminate these round-trips, collapsing multiple PLAN steps into one.

We implemented `fetch_db_context()`, which prefetches sites, assets, per-asset sensors, and failure modes before planning. The prefetched context is injected as a structured string into the planner system prompt.

In practice, optimization produced mixed results (Section V). The prefetch adds ~ 3 s overhead; this is profitable only when the EXECUTE phase would otherwise exceed ~ 60 s.

Critically, the prefetch populated only Chiller data; scenarios involving Wind Turbines at site POKMAIN received incorrect context, causing the planner to skip tool calls entirely and reducing accuracy from 0.60 to 0.43 on affected scenarios.

C. Quantization-Aware Model Substitution

1) *Motivation:* The `_call_relevancy` prompt is a binary classifier:

Given asset $\langle X \rangle$ with failure mode $\langle FM \rangle$ and sensor $\langle S \rangle$, does sensor S detect FM ? Respond YES or NO.

We hypothesised that this low-complexity task does not require 70B parameters and that a small quantized local model could match or exceed cloud API accuracy while eliminating rate-limit exposure.

2) *Experimental Design:* We evaluated five open-source model families at three precision levels — FP16 (original weights), INT8 (Q8_0), and INT4 (Q4_K_M or Q4_0) — yielding 15 model-precision configurations. All models were served via Ollama on Apple Silicon (CPU-only inference) and routed through LiteLLM. Each configuration was evaluated on all 20 FMSR benchmark scenarios (ids 101–120), producing 300 total runs. The five families are: Meta Llama 3.2 3B [9], IBM Granite 3.2 8B [10], DeepSeek-R1 7B (Qwen distill) [11], Alibaba Qwen2.5 7B [12], and Google Gemma2 9B [13].

Note that FP16 represents the *original* trained-weight precision; INT8 and INT4 are quantized compressions of it.

V. EXPERIMENTAL EVALUATION

A. Setup

All experiments ran on an Apple Silicon MacBook (CPU-only inference, unified memory architecture) against a WatsonX free-tier account. The benchmark suite covers 20 FMSR utterances from the HuggingFace `ibm-research/AssetOpsBench` dataset: Chiller 6 and Wind Turbine scenarios at site MAIN (ids 101–120).

Three measurement campaigns were conducted:

- 1) **Strategy sweep.** 15 FMSR mapping scenarios (IDs 106–120, the heavy-mapping subset), four strategies, three runs per cell with randomised strategy order per scenario. Total: 180 cells.
- 2) **Prefetch experiment.** 22 scenarios (IDs 101–122), baseline vs prefetch, three runs each, sequential mapping held fixed. Total: 132 cells.
- 3) **Quantization study.** 15 (model, precision) configurations $\times$ 20 scenarios $\times$ 1 run = 300 cells.
- 4) **System-level direct-vs-MCP.** 20 utterances run end-to-end through the in-process direct path (47 records) and the FastMCP path (42 records), scored by LLM-judge.

Metric. Tool-call accuracy measures whether the agent invoked the correct tool sequence as specified by the dataset ground truth. It captures routing correctness but not answer quality.

TABLE I
WALL TIME (SECONDS) BY PARALLELIZATION STRATEGY. SPEEDUP VS. SEQUENTIAL.

Scenario	Seq.	Parallel	Adaptive	Hedged
106	44 s	22 s	12 s	8.8 s
108	248 s	140 s	68 s	35.8 s
109	566 s	280 s	95 s	41.6 s
110	481 s	240 s	78 s	17.1 s
112	95 s	32 s	4.8 s	9.2 s
114	559 s	270 s	88 s	20.0 s
Best speedup	—	$\sim 2\times$	$\sim 20\times$	$\sim 28\times$

TABLE II
WALL TIME (SECONDS): NO CACHE VS. DB CONTEXT PREFETCH.

Scenario	No Cache	Prefetch	Δ
120	317.4 s	55.5 s	5.7 $\times$ faster
118	182.8 s	61.1 s	3.0 $\times$ faster
112	436.0 s	168.2 s	2.6 $\times$ faster
107	42.1 s	239.8 s	5.7 $\times$ slower
117	38.8 s	436.5 s	11.2 $\times$ slower
119	52.7 s	327.2 s	6.2 $\times$ slower

B. Parallelization Results

Table I summarises wall time for representative scenarios across all four strategies. Hedging dominates on tail-bound scenarios — those where sequential execution stalls on rate-limit delays.

Hedged execution yields up to $28\times$ speedup on scenarios 110 and 114. Adaptive ceiling-start is the best strategy on scenario 112 (4.8s), which has fewer than 15 calls — a case where the duplicate-request cost does not amortise. The hedging timeout of 8s was selected empirically as the 99th-percentile of non-stalled call latency; stalls begin at ~ 30 s in practice.

C. Caching Results

Table II shows wall time for the no-cache and prefetch-cache conditions across selected scenarios. Caching produces large wins on scenarios where EXECUTE is otherwise slow (>200 s) and regressions on scenarios that are already fast.

The regression on scenarios 107, 117, and 119 traces to two factors: (1) the 3 s prefetch overhead exceeds the marginal benefit on scenarios with short $N\times M$ matrices, and (2) incorrect context injection (Chiller data presented to Wind Turbine scenarios) caused the planner to omit tool calls, artificially increasing latency by forcing a re-plan cycle. A cost-gated cache — one that consults the prefetch only when predicted LLM call cost exceeds a threshold — would recover the wins without the regressions.

D. Quantization Results

1) *Aggregate Accuracy and Latency:* Table III reports average tool-call accuracy and latency across 20 scenarios for all 15 model-precision configurations. The WatsonX

TABLE III
QUANTIZATION RESULTS: ACCURACY, LATENCY, MEMORY ACROSS 20
FMSR SCENARIOS.

Model	Prec.	Acc.	Lat. (s)	Mem.
<i>WatsonX baseline (cloud API)</i>				
llama-3-3-70b	cloud	0.660	7.78	—
<i>Local models (Ollama, CPU)</i>				
Llama 3.2 3B	INT4	0.675	8.09	2.0 GB
Llama 3.2 3B	FP16	0.650	7.79	6.0 GB
Llama 3.2 3B	INT8	0.617	7.99	3.4 GB
Qwen2.5 7B	INT4	0.642	8.04	4.7 GB
Qwen2.5 7B	INT8	0.633	7.80	8.1 GB
Qwen2.5 7B	FP16	0.633	7.79	15 GB
Granite 3.2 8B	INT4	0.633	7.93	4.9 GB
Granite 3.2 8B	INT8	0.633	7.72	8.5 GB
Granite 3.2 8B	FP16	0.633	7.79	16 GB
DeepSeek-R1 7B	INT4	0.633	7.75	4.7 GB
DeepSeek-R1 7B	INT8	0.633	7.86	8.1 GB
DeepSeek-R1 7B	FP16	0.617	7.77	15 GB
Gemma2 9B	INT4	0.633	7.84	5.4 GB
Gemma2 9B	INT8	0.633	7.94	9.8 GB
Gemma2 9B	FP16	0.633	8.11	18 GB

llama-3-3-70b-instruct cloud baseline (0.660) from Fix 3 is included for reference.

2) *Key Findings:* **Finding 1: INT4 is Pareto-optimal.** Llama 3.2 3B INT4 achieves the highest accuracy (0.675) at the lowest memory (2.0 GB) and latency within 0.30 s of FP16. This holds as a trend across all five model families: INT4 is never worse than INT8 or FP16 in accuracy, and uses 67%–70% less memory than FP16.

Finding 2: INT4 outperforms FP16 (counterintuitive). For Llama 3.2 3B, INT4 beats FP16 by 2.5 pp and INT8 by 5.8 pp. Scenario 110 is the decisive case: INT4 scores 1.00, FP16 and INT8 both 0.50. For binary YES/NO classification, lower-precision weights produce more decisive output distributions, reducing hedged or ambiguous responses that propagate into incorrect tool sequences. The Q4_K_M scheme’s mixed-precision grouping preserves attention-layer signal while compressing FFN weights aggressively, which is exactly what this task needs.

Finding 3: Reasoning models are hurt by full precision. DeepSeek-R1 7B, a chain-of-thought reasoning model [11], scores 0.617 at FP16 — the *worst* result for this family. Scenario 119 is the decisive case: INT4 and INT8 both score 0.67; FP16 scores 0.33. Full-precision reasoning traces are longer, introducing intermediate steps that can contradict the final answer. Quantization acts as implicit regularisation: compressed weights limit chain-of-thought length, preventing over-reasoning on simple binary prompts.

Finding 4: Quantization-invariant models. Granite 3.2 8B and Gemma2 9B show zero accuracy variance across all three precision levels (0.633 flat). For Gemma2, FP16 is the *slowest* configuration (8.11 s) because the 18 GB weight footprint saturates Apple Silicon’s unified memory bandwidth, causing cache thrashing. INT4 is both fastest and equally accurate — making FP16 strictly dominated.

Finding 5: Local 3B model exceeds 70B cloud API. Llama 3.2 3B INT4 achieves 0.675 vs. the WatsonX llama-3-3-70b baseline of 0.660 on the same 20 scenarios: a +1.5 pp improvement. This result suggests that for binary classification sub-tasks within agentic pipelines, task complexity — not parameter count — is the binding constraint on accuracy. Replacing the cloud API with a local INT4 model eliminates rate-limit exposure (2 req/s cap), removes API key dependency, and enables 3× concurrent replicas within the same 6 GB memory budget previously required for a single FP16 copy.

Finding 6: Parameter count does not determine accuracy. Gemma2 9B, the largest model tested, matches Granite 3.2 8B and DeepSeek-R1 7B at 0.633 — no improvement over models with 2–6B fewer parameters. The accuracy ceiling at approximately 0.633–0.675 appears to be set by the planner’s tool-selection logic (driven by the 70B orchestrator LLM), not by the FMSR sub-model’s capacity.

E. Direct vs MCP, System Level

On the 20-utterance system experiment scored by LLM-judge: the in-process direct path produced 47 records (12 Pass, 14 Fail, 21 Skipped) for 46.2% accuracy over Pass plus Fail. The FastMCP path produced 42 records (22 Pass, 9 Fail, 10 Skipped, 1 Error) for 71.0%. The 24.8-pp gain is dominated by the Skipped count: the direct path Skips 21 of 47 because the trajectory crashed before producing a `final_answer`; MCP Skips only 10 of 42. Both paths use the same planner, summarizer, and judge, so the gap is not a reasoning-quality effect.

VI. DISCUSSION

A. The Structure of the Bottleneck

Profiling reveals that over 95% of total wall time is consumed by the EXECUTE phase — specifically by the `N×M_call_relevancy` matrix. The PLAN phase (LLM plan generation, ~21 s) and SUMMARIZE phase (~9 s) are effectively constants. All three optimization tracks target the EXECUTE phase through different mechanisms: parallelization reduces serialization, caching eliminates redundant calls, and quantization replaces the high-latency cloud API with a low-latency local model.

B. Where the MCP Accuracy Gain Came From

The 24.8-pp accuracy gap is significant, and it does not result from anything to do with augmented reasoning capacity. FastMCP wraps tool errors as structured JSON, which the executor reads and continues from. The in-process baseline raises Python exceptions that abort the trajectory before SUMMARIZE runs. This results in the 21 Skipped on direct, versus 10 on MCP. The MCP path finishes more trajectories, and MCP thus can be viewed as a correctness component, even if it costs extra latency and token usage.

C. Inverted AIMD vs Classical AIMD

Classical AIMD assumes a long-running flow where additive-up has wall time to converge. Plan-execute LLM workloads are bursty and short. A 54-pair mapping call finishes in tens of seconds. Probing upward in three-success steps is itself wall on the critical path, and most workloads finish before the probe ever reaches a useful limit. The inverted form gets the same multiplicative decrease for free, with no ramp-up tax. The cost is a higher chance of first-wave errors when the back-end is already overloaded; in our measurements that did not happen often enough to outweigh the wall-time win.

D. Hedging Caps p95 at a Known Cost

Hedged requests are nearly free in this regime. Inference is idempotent, the workload is back-end-bound rather than CPU-bound, and a duplicate that returns later is silently dropped. The cost is one extra request per stall event. The benefit is a hard cap on the per-call right-tail, which is what dominates the wall of any concurrent strategy (the slowest call sets the wall). For $N = 54$, even one tail event becomes the wall; hedging removes that risk for the price of $\sim 5\text{--}15\%$ extra tokens on stalled calls.

E. Combining Optimization Tracks

The three tracks are orthogonal and composable. An INT4 local model eliminates WatsonX rate limits, making hedging unnecessary and adaptive parallelization more effective (no 429 errors to recover from). A cost-gated cache would further reduce call counts. The full combination — parallelized, locally-served INT4 model with selective caching — is left as future work but is expected to compound the individual gains.

F. Threats to Validity

Single-run quantization. Quantization results use one run per (model, precision, scenario) cell. Per-scenario accuracy standard deviation is approximately ± 0.20 given the binary scoring; confidence intervals require 3+ runs per cell.

CPU-only inference. All local model inference ran on Apple Silicon CPU. GPU inference with INT4 (CUDA/Metal kernel support) would show larger latency gaps between precisions, as INT4 has dedicated kernel acceleration unavailable in CPU-bound inference.

WatsonX backend variability. Parallelization results depend on WatsonX backend load; absolute latencies are not reproducible across time windows. Relative rankings between strategies are robust.

Evaluation metric scope. Tool-call accuracy measures routing correctness but not answer quality. Relevancy matrix precision/recall — the ground-truth metric for individual `_call_relevancy` YES/NO decisions — requires manually annotated (sensor, failure mode) labels not provided in the dataset.

VII. CONCLUSION

We presented a performance engineering study of a plan-execute MCP agent for industrial asset diagnostics, identifying the $N \times M$ LLM call matrix within the FMSR tool server as the dominant bottleneck. Three optimization strategies were investigated. Speculative hedged execution reduced worst-case wall time by up to $28\times$, caching produced scenario-dependent speedups of up to $5.7\times$ with regressions on fast scenarios, and quantization-aware model substitution revealed that INT4 precision is Pareto-optimal across all five tested model families on this binary classification sub-task.

The most counterintuitive finding is that a 3B-parameter INT4 model served locally on CPU outperforms the default 70B-class cloud API by 1.5 pp in tool-call accuracy — demonstrating that agentic sub-tasks with low intrinsic complexity are better served by small, fast, local models than by large cloud APIs burdened with rate limits and network overhead. We release our instrumentation, benchmark harness, and analysis pipeline on the performance-profiling branch of the public repository.

ACKNOWLEDGMENTS

The authors thank Dr. Kaoutar El Maghraoui for guidance throughout this project and IBM Research for the public release of the AssetOpsBench dataset and benchmark infrastructure.

REFERENCES

- [1] IBM Research, “AssetOpsBench: A Benchmark for Industrial Asset Operations Agents,” *HuggingFace Datasets*, 2024. <https://huggingface.co/datasets/ibm-research/AssetOpsBench>
- [2] Anthropic, “Model Context Protocol,” Technical Specification, 2024. <https://modelcontextprotocol.io>
- [3] S. Yao *et al.*, “ReAct: Synergizing Reasoning and Acting in Language Models,” in *Proc. ICLR*, 2023.
- [4] L. Wang *et al.*, “Plan-and-Solve Prompting: Improving Zero-Shot Chain-of-Thought Reasoning by Large Language Models,” in *Proc. ACL*, 2023.
- [5] T. Dettmers *et al.*, “LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale,” in *Proc. NeurIPS*, 2022.
- [6] E. Frantar *et al.*, “GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers,” *arXiv:2210.17323*, 2022.
- [7] G. Gerganov *et al.*, “llama.cpp: Efficient LLM Inference in C/C++,” GitHub Repository, 2023. <https://github.com/ggerganov/llama.cpp>
- [8] J. Dean and L. A. Barroso, “The Tail at Scale,” *Commun. ACM*, vol. 56, no. 2, pp. 74–80, Feb. 2013.
- [9] Meta AI, “The Llama 3 Herd of Models,” *arXiv:2407.21783*, 2024.
- [10] IBM Research, “Granite 3.2: Advancing Small Language Model Capabilities,” Technical Report, IBM, 2025.
- [11] DeepSeek-AI, “DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning,” *arXiv:2501.12948*, 2025.
- [12] Qwen Team, “Qwen2.5 Technical Report,” *arXiv:2412.15115*, 2024.
- [13] Google DeepMind, “Gemma 2: Improving Open Language Models at a Practical Size,” *arXiv:2408.00118*, 2024.